IranNews - Plateforme d'actualités

Un projet PHP classique avec architecture MVC (Model-View-Controller) sans framework. Le projet est organisé en **deux applications distinctes** : une application **frontoffice** (site public) et une application **backoffice** (administration), optimisées pour fonctionner avec Docker et Docker Compose.

📋 Caractéristiques

- Architecture MVC simple et directe avec deux applications séparées
- **Frontoffice** - Site public d'actualités
- **Backoffice** - Panel d'administration
- Authentification utilisateur (login/register)
- Gestion complète des articles (Create, Read, Update, Delete)
- Recherche et filtrage par catégorie
- Design responsive
- Support Docker/Docker Compose (deux conteneurs)
- Base de données MySQL partagée avec PDO
- Model.php comme classe de base (héritable pour tous les modèles)

🏗 Structure du projet

```
IranNews/
├── frontoffice/             # Application publique (port 8050)
│   ├── app/
│   │   ├── config/
│   │   │   ├── bootstrap.php     # Configuration et classe Database
│   │   │   ├── database.php      # Paramètres de connexion
│   │   │   └── helpers.php       # Fonctions utilitaires
│   │   ├── controllers/
│   │   │   └── FrontController.php   # Gestion des pages publiques
│   │   ├── models/
│   │   │   ├── Category.php      # Modèle des catégories
│   │   │   ├── Model.php         # Classe de base des modèles
│   │   │   ├── News.php          # Modèle des articles
│   │   │   └── User.php          # Modèle des utilisateurs
│   │   ├── route/
│   │   │   └── routes.php        # Définition des routes
│   │   └── views/
│   │       ├── front/            # Pages publiques (home, category, etc.)
│   │       ├── errors/           # Pages d'erreur (404, 500)
│   │       ├── news/             # Vues articles
│   │       └── layouts/
│   │           └── app.php       # Layout principal
│   └── public/
│       ├── index.php             # Point d'entrée du frontoffice
│       ├── robots.txt
│       ├── sitemap.php
```

```
│       └── assets/
│           ├── css/style.css     # Styles publics
│           ├── js/script.js      # Scripts publics
│           └── tinymce/          # Éditeur WYSIWYG
│
├── backoffice/               # Application d'administration (port 8051)
│   ├── app/
│   │   ├── config/
│   │   │   ├── bootstrap.php     # Configuration et classe Database
│   │   │   ├── database.php      # Paramètres de connexion
│   │   │   └── helpers.php       # Fonctions utilitaires
│   │   ├── controllers/
│   │   │   ├── AuthController.php    # Authentification Admin
│   │   │   ├── BackController.php    # Gestion administrative
│   │   │   └── CategoryController.php # Gestion des catégories
│   │   ├── models/
│   │   │   ├── Category.php      # Modèle des catégories
│   │   │   ├── Model.php         # Classe de base des modèles
│   │   │   ├── News.php          # Modèle des articles
│   │   │   └── User.php          # Modèle des utilisateurs
│   │   ├── route/
│   │   │   └── routes.php        # Définition des routes
│   │   └── views/
│   │       ├── auth/             # Pages de connexion/inscription
│   │       ├── back/             # Interface administrative
│   │       │   ├── dashboard.php
│   │       │   ├── categories/   # Gestion des catégories
│   │       │   ├── news/         # Gestion des articles
│   │       │   └── users/        # Gestion des utilisateurs
│   │       ├── errors/           # Pages d'erreur
│   │       ├── news/             # Vues articles partagées
│   │       └── layouts/
│   │           └── back.php      # Layout administratif
│   └── public/
│       ├── index.php             # Point d'entrée du backoffice
│       └── assets/
│           ├── css/style.css     # Styles admin
│           ├── js/script.js      # Scripts admin
│           └── tinymce/          # Éditeur WYSIWYG
│
├── public/                   # Application racine (optionnel)
│   ├── index.php
│   ├── robots.txt
│   ├── sitemap.php
│   └── uploads/
│       └── articles/         # Stockage des images des articles
│
├── docker/                   # Configuration Docker
│   ├── apache/               # Configuration Apache
│   └── mysql/
│       └── init/
│           └── 01-init.sql   # Script d'initialisation DB
│
```

```
├── Dockerfile                # Configuration Docker multi-conteneurs
├── docker-compose.yml        # Orchestre frontoffice, backoffice et DB
├── composer.json             # Dépendances PHP (si utilisé)
└── README.md
```

🚀 Démarrage avec Docker

Prérequis

- Docker
- Docker Compose

Installation et lancement

1. **Cloner/placer le projet**

```
cd IranNews
```

2. **Lancer le projet**

```
docker-compose up -d
```

3. **Accéder à l'application**

Application	URL	Description
Frontoffice	http://localhost:8050	Site public d'actualités
Backoffice	http://localhost:8051	Panel d'administration
MySQL	localhost:3306	Base de données (via dbclient)

4. **Arrêter le projet**

```
docker-compose down
```

📝 Identifiants de test

Admin:

- Utilisateur: `admin`
- Mot de passe: `admin123`

🔄 Pages et Routes disponibles

Frontoffice (Application publique - port 8050)

Routes définies dans `frontoffice/app/route/routes.php` :

- `/` ou `?page=accueil` - Accueil avec liste des articles
- `?page=news&id={ID}` ou `/article/{slug}` - Détail d'un article
- `?page=search&q=QUERY` ou `/search?q=...` - Recherche d'articles
- `?page=category&id={ID}` ou `/category/{slug}` - Articles par catégorie
- `?page=about` - À propos
- `?page=contact` - Contact
- `?page=login` - Connexion utilisateur
- `?page=register` - Inscription utilisateur

Backoffice (Administration - port 8051)

Routes définies dans `backoffice/app/route/routes.php` :

Authentification:

- `/login` - Connexion administrateur
- `/register` - Inscription administrateur
- `/logout` - Déconnexion

Tableau de bord:

- `/` ou `?page=dashboard` - Tableau de bord administrateur

Gestion des articles:

- `/news` ou `?page=news` - Liste des articles
- `/news/create` ou `?page=news&action=create` - Créer un article
- `/news/{id}/edit` ou `?page=news&action=edit&id={id}` - Éditer un article
- `/news/{id}/show` ou `?page=news&action=show&id={id}` - Voir un article
- `/news/{id}/delete` ou `?page=news&action=delete&id={id}` - Supprimer un article

Gestion des catégories:

- `/categories` - Liste des catégories
- `/categories/create` - Créer une catégorie
- `/categories/{id}/edit` - Éditer une catégorie

Gestion des utilisateurs:

- `/users` - Liste des utilisateurs
- `/users/{id}/edit` - Éditer un utilisateur
- `/users/{id}/delete` - Supprimer un utilisateur

Base de données

La base de données MySQL est **partagée** entre les deux applications (frontoffice et backoffice).

Tables créées (via `docker/mysql/init/01-init.sql`)

- `users` - Utilisateurs (id, username, email, password, role, created_at)
- `news` - Articles (id, title, slug, content, user_id, category_id, image_url, published, views, created_at, updated_at)
- `categories` - Catégories (id, name, slug, description, created_at)
- `comments` - Commentaires (id, news_id, user_id, content, approved, created_at)

Variables d'environnement (Docker)

```
DB_HOST=db                  # Hôte MySQL (service Docker)
DB_USER=irannews_user       # Utilisateur MySQL
DB_PASSWORD=irannews_pass   # Mot de passe
DB_NAME=irannews            # Nom de la base
DB_PORT=3306                # Port MySQL
FRONT_BASE_URL=http://localhost:8050
BACK_BASE_URL=http://localhost:8051
```

🔐 Authentification

L'authentification utilise PHP sessions et password_hash/password_verify pour sécuriser les mots de passe.

Rôles disponibles

- `admin` - Accès complet
- `author` - Créer et éditer ses articles
- `user` - Accès à la lecture seule

💾 Configuration

Avec Docker (recommandé)

Les paramètres sont définis dans `docker-compose.yml` :

- Les variables d'environnement sont injectées dans les deux conteneurs
- La base de données MySQL est initialisée automatiquement
- Les deux applications partagent la même base

Sans Docker (développement local)

1. **Prérequis** - PHP 8.2+ avec extensions :
   - `pdo_mysql`
   - `gd` (pour les images)
2. **Configuration** - Éditer les fichiers:
   - `frontoffice/app/config/database.php`
   - `backoffice/app/config/database.php`

3. **Base de données** - Importer le schéma:

```
mysql -u root -p irannews < docker/mysql/init/01-init.sql
```

4. **Lancer le frontoffice** (port 8050):

```
cd frontoffice
php -S localhost:8050 -t public/
```

5. **Lancer le backoffice** (port 8051, dans un autre terminal):

```
cd backoffice
php -S localhost:8051 -t public/
```

Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                      Docker Compose                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   ┌──────────────────────┐    ┌──────────────────────┐       │
│   │   FRONTOFFICE        │    │   BACKOFFICE         │       │
│   │   (localhost:8050)   │    │   (localhost:8051)   │       │
│   │                      │    │                      │       │
│   │  - Site public       │    │  - Administration    │       │
│   │  - Articles          │    │  - Gestion News      │       │
│   │  - Recherche         │    │  - Gestion Users     │       │
│   │  - Catégories        │    │  - Gestion Catégories│       │
│   └──────────┬───────────┘    └──────────┬───────────┘       │
│              │                           │                    │
│              └───────────┬───────────────┘                    │
│                          │                                    │
│                    ┌─────▼──────┐                            │
│                    │   MySQL 8.0    │                         │
│                    │   (Port 3306)  │                         │
│                    │                │                         │
│                    │ Base partagée  │                         │
│                    │   'irannews'   │                         │
│                    └────────────────┘                         │
└──────────────────────────────────────────────────────────────┘
```

Points clés

- **Deux conteneurs Apache/PHP** - Frontoffice et Backoffice sont indépendants
- **Une base de données partagée** - MySQL centralisé pour les deux applications
- **Variables d'environnement** - Configuration via `docker-compose.yml`
- **Volumes montés** - Développement direct sans rebuild à chaque changement
- **Architecture sans framework** - PHP pur, MVC simple et lisible

Fichiers principaux

Points d'entrée

- `frontoffice/public/index.php` - Point d'entrée de l'application publique
- `backoffice/public/index.php` - Point d'entrée de l'administration

Configuration commune

- `frontoffice/app/config/bootstrap.php` - Classe Database et initialisations
- `backoffice/app/config/bootstrap.php` - Classe Database et initialisations
- `frontoffice/app/config/database.php` - Paramètres MySQL (frontoffice)
- `backoffice/app/config/database.php` - Paramètres MySQL (backoffice)

Dockerization

- `Dockerfile` - Image Apache + PHP pour les deux applications
- `docker-compose.yml` - Orchestration des deux conteneurs + DB
- `docker/apache/` - Configuration Apache (mod_rewrite, vhosts)
- `docker/mysql/init/01-init.sql` - Schéma et données initiales

Controllers

- `frontoffice/app/controllers/FrontController.php` - Pages publiques
- `backoffice/app/controllers/AuthController.php` - Auth admin
- `backoffice/app/controllers/BackController.php` - Admin principal
- `backoffice/app/controllers/CategoryController.php` - Gestion catégories

Models

- `frontoffice/app/models/` / `backoffice/app/models/` - Identiques (partagés logiquement)
  - `Model.php` - Classe de base (PDO)
  - `News.php` - Modèle articles
  - `User.php` - Modèle utilisateurs
  - `Category.php` - Modèle catégories

⚠ Points importants

- **Deux applications indépendantes** - Frontoffice et Backoffice sont deux projets PHP séparés
- **Modèles partagés** - Les fichiers modèles sont en double (identical) dans les deux dossiers `app/models/`
- **PDO avec prepared statements** - Sécurité SQL, protection contre les injections
- **Classe Model.php** - Base héritable pour tous les modèles (News, User, Category)

- **Authentication via sessions PHP** - password_hash/password_verify pour les mots de passe
- **Pas de dépendances externes** - Framework-free, PHP pur (sauf PHP-FIG standards)
- **Routes simple** - Routage par paramètres GET ou RewriteRule Apache

📦 Extensibilité

Ajouter une page au Frontoffice

1. Créer une méthode dans `frontoffice/app/controllers/FrontController.php`
2. Ajouter une route dans `frontoffice/app/route/routes.php`
3. Créer la vue dans `frontoffice/app/views/front/`
4. Ajouter le lien dans le menu du layout `frontoffice/app/views/layouts/app.php`

Ajouter une fonctionnalité au Backoffice

1. Créer une méthode dans le controller approprié (BackController.php, CategoryController.php, etc.)
2. Ajouter une route dans `backoffice/app/route/routes.php`
3. Créer les vues dans `backoffice/app/views/back/`
4. Ajouter le lien dans la navigation du layout `backoffice/app/views/layouts/back.php`

Créer un nouveau modèle

1. Créer une classe héritant de `Model.php` dans `app/models/`
2. Définir les propriétés et les méthodes de accès à la base
3. Utiliser PDO avec prepared statements

Exemple :

```php
class Article extends Model {
    protected $table = 'news';

    public function __construct($db) {
        parent::__construct($db, $this->table);
    }
}
```

🐛 Dépannage

Problèmes Docker

Erreur de connexion à la base de données

```bash
# Vérifier que MySQL est bien lancé
docker-compose ps

# Vérifier les logs MySQL
docker-compose logs db
```

```
# Vérifier les variables d'environnement
docker-compose config | grep DB_
```

Les deux applications ne se chargent pas

```
# Reconstruire les images
docker-compose down
docker-compose up -d --build

# Vérifier que Apache est en écoute sur les bons ports
docker-compose logs frontoffice
docker-compose logs backoffice
```

Les uploads d'images ne fonctionnent pas

```
# Vérifiez les permissions du dossier uploads
chmod 755 public/uploads/articles
chmod 755 public/uploads/articles/*
```

Problèmes généraux

Page non trouvée (404)

- Vérifier que le paramètre `page` est correct
- Vérifier que la route existe dans `app/route/routes.php`
- Vérifier que le fichier de vue existe

Connexion refusée MySQL

- Vérifier les paramètres dans `app/config/database.php`
- Vérifier que le mot de passe est correct dans `docker-compose.yml`
- Vérifier que le conteneur MySQL est en cours d'exécution

Erreurs de session

- Vérifier que `session_start()` est appelé en haut de `bootstrap.php`
- Vérifier que le dossier temp de PHP a les bonnes permissions

Logs recommandés

```
# Logs des deux conteneurs
docker-compose logs -f frontoffice
docker-compose logs -f backoffice
docker-compose logs -f db
```

```
# Logs du système hôte
tail -f /var/log/apache2/error.log  # Sur le serveur local
tail -f /var/log/mysql/error.log    # Sur le serveur local
```

📜 Licence

Projet pédagogique - S6 Web Avancé

🎯 Roadmap

- ☐ Synchroniser les modèles entre frontoffice et backoffice (DRY)
- ☐ API REST pour communication frontoffice/backoffice
- ☐ Implémenter les commentaires d'articles
- ☐ Ajouter la pagination côté admin
- ☐ Système de cache (Redis optionnel)
- ☐ Upload d'images avec validation
- ☐ Swagger/OpenAPI pour l'API
- ☐ Tests unitaires (PHPUnit)
- ☐ Multi-langue (i18n)
- ☐ Export PDF des articles
- ☐ Analytics/Statistiques
- ☐ Migrer vers un Mini-framework (Laravel, Symfony Mini)

Version: 1.0.0
Dernière mise à jour: Mars 2026
Auteur: Classe S6 Web Avancé